

Exercise 1: Control Structures

Creating Schema, Tables and Inserting Values

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'bank_db' selected. The main editor shows a SQL script with the following queries:

```
VALUES
(101,1,8.5,CURDATE()+INTERVAL 10 DAY),
(102,2,9.8,CURDATE()+INTERVAL 50 DAY),
(103,3,7.5,CURDATE()+INTERVAL 20 DAY),
(104,4,10.8,CURDATE()+INTERVAL 5 DAY);

Select * from Customers;
select * from Loans;
```

The 'Result Grid' shows the output of the queries:

CustomerID	CustomerName	Age	Balance	ISVP
1	Jhon	65	15000.00	0
2	Mary	64	8000.00	0
3	David	35	25000.00	0
4	Sarah	75	5000.00	0

The 'Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
1	22:47:21	CREATE DATABASE bank_db	1 row(s) affected	0.000 sec
2	22:47:27	USE bank_db	0 row(s) affected	0.000 sec
3	22:47:39	CREATE TABLE Customers (CustomerID INT PRIMARY KEY, CustomerName VARCHAR(100), Age I...	0 row(s) affected	0.016 sec
4	22:47:44	CREATE TABLE Loans (LoanID INT PRIMARY KEY, CustomerID INT, InterestRate DECIMAL(5,2...	0 row(s) affected	0.031 sec
5	22:47:50	Insert into Customers values (1,Jhon,65,15000,false), (2,Mary,64,8000,false), (3,David,35,25000,false), (4...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec
6	22:47:57	INSERT INTO Loans VALUES (101,1,8.5,CURDATE()+INTERVAL 10 DAY), (102,2,9.8,CURDATE()+INT...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.015 sec
7	22:48:01	Select * from Customers LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

OUTPUT – Scenario 1

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'bank_db' selected. The main editor shows a SQL script with the following queries:

```
END IF;
END LOOP;
CLOSE customer_cursor;
END //
DELIMITER ;
```

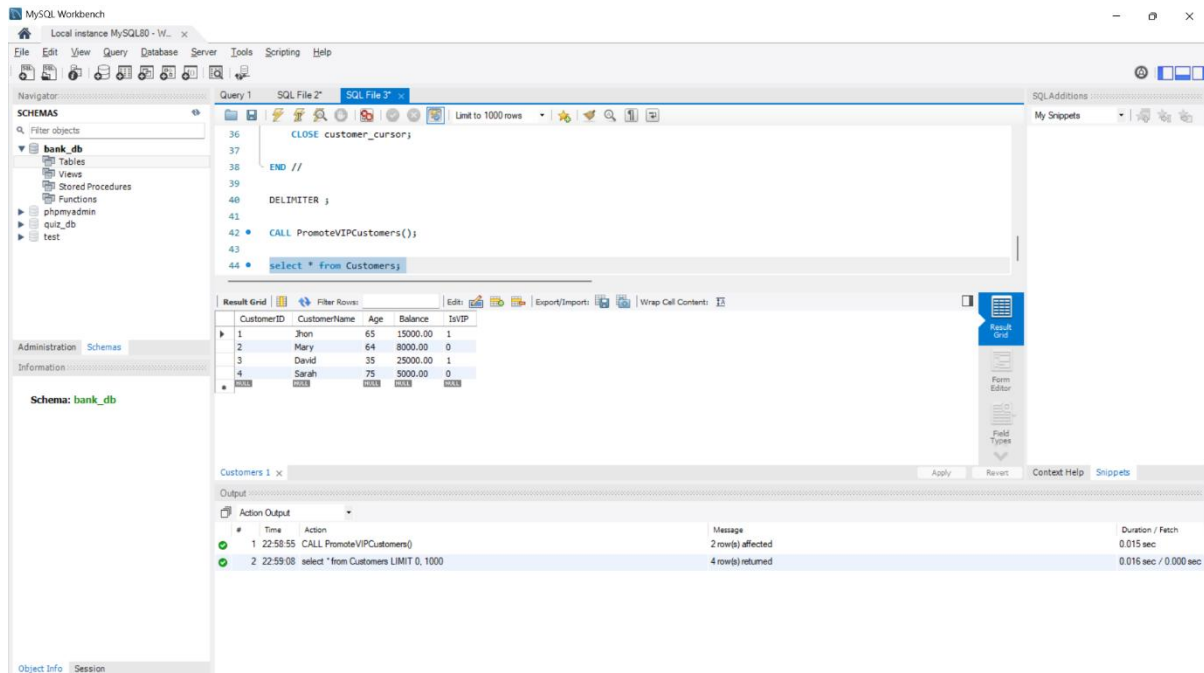
The 'Result Grid' shows the output of the queries:

LoanID	CustomerID	InterestRate	DueDate
101	1	7.50	2026-06-30
102	2	8.00	2026-08-09
103	3	7.50	2026-07-10
104	4	9.00	2026-06-25

The 'Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
1	22:52:35	CREATE PROCEDURE ApplySeniorDiscount() BEGIN DECLARE done INT DEFAULT FALSE; DECL...	0 row(s) affected	0.016 sec
2	22:52:22	SELECT * FROM Loans LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
3	22:52:27	CALL ApplySeniorDiscount()	3 row(s) affected	0.015 sec
4	22:53:45	SELECT * FROM Loans LIMIT 0, 1000	4 row(s) returned	0.016 sec / 0.000 sec

OUTPUT - Scenario 2



MySQL Workbench interface showing the execution of a SQL script for Scenario 2. The script includes a cursor, a loop, and a call to a stored procedure. The output shows the execution of the stored procedure and a subsequent select statement.

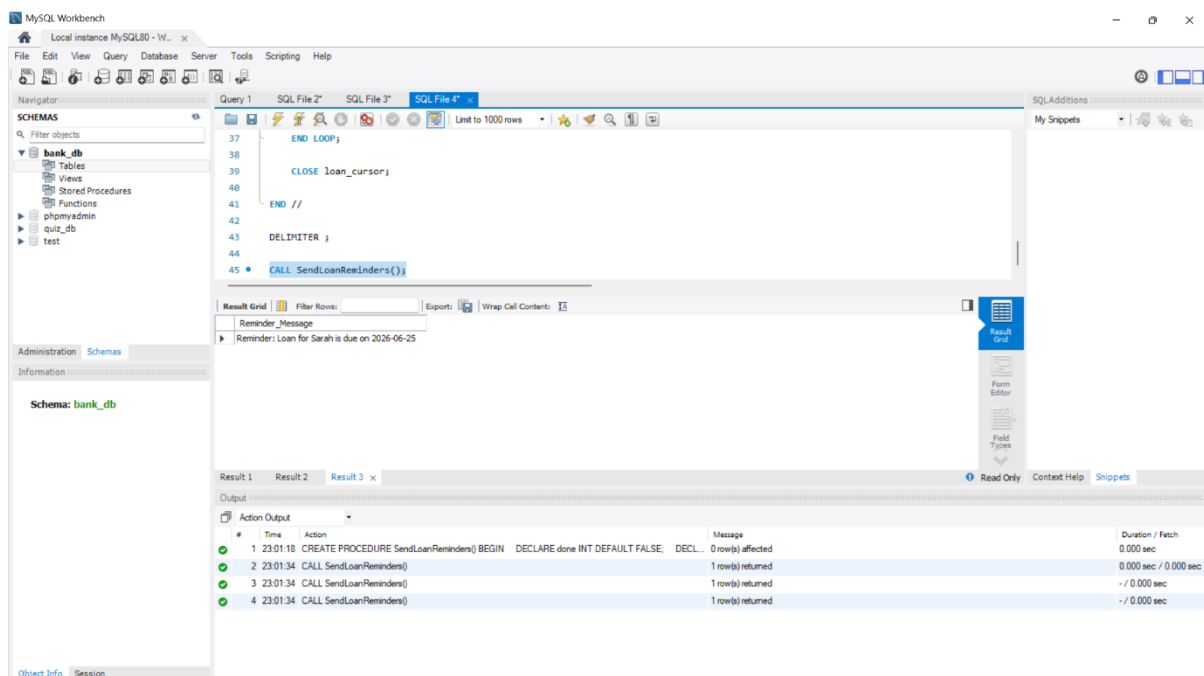
```
36 CLOSE customer_cursor;
37
38 END //
39
40 DELIMITER ;
41
42 CALL PromoteVIPCustomers();
43
44 select * from Customers;
```

CustomerID	CustomerName	Age	Balance	IsVIP
1	Jhon	65	15000.00	1
2	Mary	64	8000.00	0
3	David	35	25000.00	1
4	Sarah	75	5000.00	0

Customers 1 x

#	Time	Action	Message	Duration / Fetch
1	22:58:55	CALL PromoteVIPCustomers()	2 row(s) affected	0.015 sec
2	22:59:08	select * from Customers LIMIT 0, 1000	4 row(s) returned	0.016 sec / 0.000 sec

OUTPUT – Scenario 3



MySQL Workbench interface showing the execution of a SQL script for Scenario 3. The script includes a loop, a cursor, and a call to a stored procedure. The output shows the execution of the stored procedure and a subsequent select statement.

```
37 END LOOP;
38
39 CLOSE loan_cursor;
40
41 END //
42
43 DELIMITER ;
44
45 CALL SendLoanReminders();
```

Reminder_Message

Reminder: Loan for Sarah is due on 2026-06-25

Result 1	Result 2	Result 3

#	Time	Action	Message	Duration / Fetch
1	23:01:18	CREATE PROCEDURE SendLoanReminders() BEGIN DECLARE done INT DEFAULT FALSE; DECL...	0 row(s) affected	0.000 sec
2	23:01:34	CALL SendLoanReminders()	1 row(s) returned	0.000 sec / 0.000 sec
3	23:01:34	CALL SendLoanReminders()	1 row(s) returned	~ / 0.000 sec
4	23:01:34	CALL SendLoanReminders()	1 row(s) returned	~ / 0.000 sec

Exercise 3: Stored Procedures

Using the prev “bank_db” and inserting values to new table “Accounts”

The screenshot shows the MySQL Workbench interface with the following details:

- Navigator:** Schemas list includes bank_db, Tables, Views, Stored Procedures, Functions, phpmyadmin, quit_db, and test.
- SQL Editor:** Contains the following SQL script:

```
11
12 INSERT INTO Accounts
13 VALUES
14 (1001,1,'Savings',5000),
15 (1002,2,'Savings',10000),
16 (1003,3,'Current',15000),
17 (1004,4,'Savings',20000);
18
19 SELECT * FROM Accounts;
```
- Result Grid:** Displays the data inserted into the Accounts table:

AccountID	CustomerID	AccountType	Balance
1001	1	Savings	5000.00
1002	2	Savings	10000.00
1003	3	Current	15000.00
1004	4	Savings	20000.00
- Output:** Shows the execution log with the following actions:

#	Time	Action	Message	Duration / Fetch
1	23:15:38	USE bank_db	0 row(s) affected	0.016 sec
2	23:15:42	CREATE TABLE Accounts (AccountID INT PRIMARY KEY, CustomerID INT, AccountType VARC...	0 row(s) affected	0.031 sec
3	23:15:46	INSERT INTO Accounts VALUES (1001,1,'Savings',5000), (1002,2,'Savings',10000), (1003,3,'Current',150...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec
4	23:15:50	SELECT * FROM Accounts LIMIT 0, 1000	4 row(s) returned	0.016 sec / 0.000 sec

OUTPUT Scenario - 1

The screenshot shows the MySQL Workbench interface with the following details:

- Navigator:** Schemas list includes bank_db, Tables, Views, Stored Procedures, Functions, phpmyadmin, quit_db, and test.
- SQL Editor:** Contains the following SQL script:

```
8
9 WHERE AccountType = 'Savings';
10
11 END //
12
13 DELIMITER ;
14
15 SET SQL_SAFE_UPDATES = 0;
16 CALL ProcessMonthlyInterest();
```
- Result Grid:** Displays the data in the Accounts table, which remains the same as in the previous screenshot:

AccountID	CustomerID	AccountType	Balance
1001	1	Savings	5000.00
1002	2	Savings	10000.00
1003	3	Current	15000.00
1004	4	Savings	20000.00
- Output:** Shows the execution log with the following actions:

#	Time	Action	Message	Duration / Fetch
2	23:15:42	CREATE TABLE Accounts (AccountID INT PRIMARY KEY, CustomerID INT, AccountType VA...	0 row(s) affected	0.031 sec
3	23:15:46	INSERT INTO Accounts VALUES (1001,1,'Savings',5000), (1002,2,'Savings',10000), (1003,3,'Current',1...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec
4	23:15:50	SELECT * FROM Accounts LIMIT 0, 1000	4 row(s) returned	0.016 sec / 0.000 sec
5	23:16:42	CREATE PROCEDURE ProcessMonthlyInterest() BEGIN UPDATE Accounts SET Balance = Balan...	0 row(s) affected	0.016 sec
6	23:16:50	CALL ProcessMonthlyInterest()	Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE that...	0.000 sec
7	23:20:28	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
8	23:20:32	CALL ProcessMonthlyInterest()	3 row(s) affected	0.015 sec
9	23:20:45	SELECT * FROM Accounts LIMIT 0, 1000	4 row(s) returned	0.016 sec / 0.000 sec

OUTPUT Scenario – 2

The screenshot shows the MySQL Workbench interface for Scenario 2. The SQL Editor contains the following queries:

```

13 WHERE Department = dept_name;
14
15 END //
16
17 DELIMITER ;
18
19 CALL UpdateEmployeeBonus('IT',10);
20
21 SELECT * FROM Employees;
  
```

The Result Grid displays the output of the last query, showing a table with 4 rows and 4 columns: EmployeeID, EmployeeName, Department, and Salary.

EmployeeID	EmployeeName	Department	Salary
1	Rahul	IT	55000.00
2	Priya	HR	45000.00
3	Arun	IT	66000.00
4	Neha	Finance	55000.00

The Output pane shows the execution log for the queries:

#	Time	Action	Message	Duration / Fetch
1	23:26:12	CREATE TABLE Employees (EmployeeID INT PRIMARY KEY, EmployeeName VARCHAR(100), ...	0 row(s) affected	0.000 sec
2	23:26:19	INSERT INTO Employees VALUES (1,'Rahul','IT',55000), (2,'Priya','HR',45000), (3,'Arun','IT',66000), (4,'Neha','Finance',55000);	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec
3	23:26:37	select * from Employees LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
4	23:26:57	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
5	23:27:20	CREATE PROCEDURE UpdateEmployeeBonus(IN dept_name VARCHAR(50), IN bonus_percent D...	0 row(s) affected	0.016 sec
6	23:27:33	CALL UpdateEmployeeBonus('IT',10)	2 row(s) affected	0.000 sec
7	23:27:51	SELECT * FROM Employees LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

OUTPUT Scenario – 3

The screenshot shows the MySQL Workbench interface for Scenario 3. The SQL Editor contains the following queries:

```

33 END IF;
34
35 END //
36
37 DELIMITER ;
38
39 CALL TransferFunds(1001,1002,1000);
40
41 SELECT * FROM Accounts;
  
```

The Result Grid displays the output of the last query, showing a table with 4 rows and 4 columns: AccountID, CustomerID, AccountType, and Balance.

AccountID	CustomerID	AccountType	Balance
1001	1	Savings	4050.00
1002	2	Savings	11100.00
1003	3	Current	15000.00
1004	4	Savings	20200.00

The Output pane shows the execution log for the queries:

#	Time	Action	Message	Duration / Fetch
3	23:26:37	select * from Employees LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
4	23:26:57	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
5	23:27:20	CREATE PROCEDURE UpdateEmployeeBonus(IN dept_name VARCHAR(50), IN bonus_percent ...	0 row(s) affected	0.016 sec
6	23:27:33	CALL UpdateEmployeeBonus('IT',10)	2 row(s) affected	0.000 sec
7	23:27:51	SELECT * FROM Employees LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
8	23:29:30	CREATE PROCEDURE TransferFunds(IN from_account INT, IN to_account INT, IN transfer_a...	0 row(s) affected	0.000 sec
9	23:29:42	CALL TransferFunds(1001,1002,1000)	1 row(s) returned	0.015 sec / 0.000 sec
10	23:30:32	SELECT * FROM Accounts LIMIT 0, 1000	4 row(s) returned	0.016 sec / 0.000 sec